Tabla de contenidos

1. Envío de datos
    a. Envío de datos con `JSON`
    b. Envío de datos con `pickle`
    c. Envío de muchos datos
2. Ejemplo completo: servidor con manejo de múltiples clientes en forma concurrente
3. Ejemplo completo: cliente conectado a más de un servidor
4. Ejemplo completo: Interacción entre clientes P2P

Ejemplos de *networking* con Python

IMPORTANTE: El siguiente material incluye ejemplos que deberían ser ejecutados en programas separados, donde uno actúa como el cliente, y otro actúa como el servidor. Para ello, copie los códigos correspondientes en archivos .py en su computador y ejecútelo en consolas separadas. Si ejecuta los códigos directamente en este jupyter-notebook los programas se pueden quedar pegados esperando una conexión.

En este *notebook*, veremos ejemplos de distintas aplicaciones de Networking con Python.

Envío de datos

En el siguiente ejemplo, veremos algunas variaciones de envíos de datos.

Envío de datos con JSON

Intenta hacer la prueba con dos terminales distintas: en uno ejecuta el código de la siguiente celda con la definición del servidor, y en el otro la celda subsiguiente con la definición del cliente que envía los datos JSON.

El primer ejemplo corresponde a cómo programar un servidor que reciba datos y los envíe de vuelta al cliente, lo que se conoce como un *echo server*.

```
# Implementación del servidor que recibe datos y los envía de vuelta.
# Esto comúnmente se denomina como 'echo server'.

import socket


host = 'localhost'   # Usar nuestro computador como host
port = 12345

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind((host, port))
print("Listo para recibir. Ahora voy a esperar que me hablen...")
```

```
sock.listen()

sock_cliente, (host_cliente, puerto_cliente) = sock.accept()
print("Conexión desde", host_cliente, puerto_cliente)

while True:
    data = sock_cliente.recv(4096)
    print(f"Recibí estos bytes: {data}")
    if not data:
        break
    print("Los enviaré de vuelta")
    sock_cliente.sendall(data)

sock_cliente.close()
sock.close()
```

El cliente envía datos JSON y los imprime una vez que el servidor se los envía de vuelta. Luego cierra la conexión.

```
# Implementación del cliente que envía los datos en formato JSON.
# Pon atención en la serialización y transformación a bytes.

import json
import socket


server_host = ''  # Debemos poner aquí la dirección IP del servidor.
# Si no ponemos nada, supone que estamos hablando con un programa en
el mismo host.
server_port = 12345

# Generamos la información que enviaremos; en este caso, es un simple
diccionario.
data = {1: "Hola", 2: "Chao"}
mensaje = json.dumps(data)

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect((server_host, server_port))

# Debemos enviar bytes, por lo que codificamos el mensaje.
sock.sendall(mensaje.encode('utf-8'))

# Decodificamos los bytes de la respuesta y luego los deserializamos
con JSON.
data = json.loads(sock.recv(4096).decode('utf-8'))
print(data)
input("ENTER para terminar.")
sock.close()
```

Notemos que cuando el cliente termina, la conexión con el servidor se cierra. Esto se traduce en que el servidor dejará de recibir datos y, de acuerdo a cómo está construido, también termina. Si queremos que el servidor quede disponible para recibir otras conexiones, debemos volver a ejecutar `listen`.

Envío de datos con `pickle`

Al igual que el ejemplo anterior, podemos enviar cualquier objeto de Python serializado con `pickle`. El siguiente código muestra un ejemplo de cómo conectarse al servidor anterior y enviarle datos serializados con `pickle`. Cuando los *bytes* vienen de vuelta desde el servidor lo deserializamos y tenemos nuevamente la instancia de la clase `Persona` que habíamos enviado.

Puedes usar este cliente con **el mismo servidor (*echo server*) del ejemplo anterior**. El *echo server* no se preocupa de qué tipo de datos recibe. Sólo recibe bytes y los manda de vuelta.

```
# Implementación del cliente que envía los datos en formato Pickle.
# Pon atención en la serialización y transformación a bytes.

import pickle
import socket


server_host = ''  # Debemos poner aquí la dirección IP del servidor.
# Si no ponemos nada, supone que estamos hablando con un programa en
el mismo host.
port = 12345


class Persona:
    def __init__(self, nombre, correo):
        self.nombre = nombre
        self.correo = correo


# Enviaremos esta instancia de la clase Persona.
persona = Persona("Juan Pérez", "jp@ejemplo.com")
mensaje = pickle.dumps(persona)

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect((server_host, port))
sock.sendall(mensaje)

data = pickle.loads(sock.recv(4096))
print(f"He recibido bytes de vuelta y los deserialicé a {type(data)}")
print(data.nombre)
sock.close()
```

Además, podríamos modificar el código del servidor para que haga alguna acción requerida con los datos que recibe y envíe el resultado de esta acción de vuelta. Por ejemplo, podríamos verificar en el servidor los datos de un cliente para iniciar sesión.

Recomendamos fuertemente, dentro de lo posible, conectar dos computadores y probar enviando hacia y desde ambas partes, de tal forma de familiarizarse lo más posible con los *sockets* y disfrutar de los beneficios que nos ofrece la red. Para ello, debes ingresar en la variable *host*, la **dirección IP del servidor** en el caso de los clientes. Los servidores no necesitan modificación.

¿Cómo conseguir la dirección IP de un computador? El método varía dependiendo de cada sistema operativo, pero usualmente hay interfaces de configuración de la red en las cuales se puede averiguar ese dato. Nada que una búsqueda específica en Internet para tu sistema operativo no pueda resolver. Atención que si preguntas en sitios como https://whatismyipaddress.com/ encontrarás la dirección pública que te asigna tu proveedor de Internet, pero si quieres conectarte con un computador que está en tu misma casa (i.e. en tu misma subred) seguramente la dirección IP será distinta pues se trata de una dirección "interna" a tu subred.

Envío de muchos datos

Supongamos que queremos transmitir un archivo relativamente grande entre dos computadores. Lo único que tenemos que hacer es leer el archivo en *bytes*, y transmitirlo con un *socket*. Implementemos el código del emisor y del receptor del archivo.

```
# Este es el receptor del archivo.
import socket


host = 'localhost'
port = 12345

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind((host, port))
sock.listen()
print("Escuchando...")

# Aceptamos un cliente.
sock_cliente, (host_cliente, port_cliente) = sock.accept()
print("Conexión entrante aceptada.")

# Leemos la información y la guardamos en un archivo.
datos = sock_cliente.recv(4096)
with open('files/recibido.bin', 'wb') as binfile:
    binfile.write(datos)

print("¡Archivo recibido!")
# Le enviamos una respuesta a la contraparte.
sock_cliente.sendall("Gracias.".encode('utf-8'))

# Cerramos los sockets.
sock_cliente.close()
sock.close()
```

```python
# Este es el emisor del archivo.
import socket


host_receptor = 'localhost'  # Si lo ejecutas en otro computador, aquí
debes poner
# debes poner su dirección IP
puerto_receptor = 12345

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Nos conectamos al receptor del archivo, que ya debería estar
escuchando.
sock.connect((host_receptor, puerto_receptor))
print("Conexión establecida.")

# Leemos el archivo y lo enviamos.
with open('files/enviar.bin', 'rb') as binfile:
    datos = binfile.read()
    sock.sendall(datos)

print("¡Archivo enviado!")

# Imprimimos lo que nos responda la contraparte.
print("Respuesta:", sock.recv(4096).decode('utf-8'))

# Cerramos el socket.
sock.close()
```

Veamos lo que sucede al tener cada celda en un programa distinto.

Todo parece ir bien pero, si comparamos los archivos, vemos diferencias significativas de tamaño.

¡Leímos muy pocos *bytes*! Podríamos cambiar el valor en `recv(buffer)`, pero si hacemos esto seguimos teniendo problemas:

- ¿Cómo saber de antemano el tamaño de lo que voy a recibir?
- E incluso sabiendo el tamaño del mensaje que voy a recibir, ¿qué ocurre si tengo que recibir *megabytes* o *gigabytes* de datos?

La solución a este problema tiene dos partes. Primero, tener un **protocolo** o esquema de comunicación para nuestra aplicación de transferencia de archivos. Por ahora, sólo necesitamos una forma de indicar el tamaño del archivo que queremos mandar. Una forma simple es enviar un número que ocupe cuatro *bytes* para indicar el tamaño del archivo, antes de enviar el archivo en sí. Si usamos 4 *bytes* como encabezado de todos los mensajes para indicar el largo de estos,

podremos enviar mensajes de hasta 2^{32} *bytes* (4 GiB). Esta es una cantidad razonable para la mayoría de los casos.

Ya sabiendo el número de *bytes* que tenemos que leer, tenemos que encargarnos de leer el archivo por *chunks*, hasta completarlo. Tomando eso en cuenta, volvamos a implementar nuestro emisor y receptor de archivos.

```
# Receptor mejorado del archivo.
import socket


host = 'localhost'
puerto = 12345

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind((host, puerto))
sock.listen()
print('Escuchando...')

# Aceptamos un cliente.
sock_cliente, (host_cliente, puerto_cliente) = sock.accept()
print("Conexión entrante aceptada.")

# Leemos primero los 4 bytes del tamaño del archivo.
# Con esto transformamos una serie de bytes en un int.
largo_archivo = int.from_bytes(sock_cliente.recv(4), byteorder='big')
datos = bytearray()
bytes_leidos = 0
print(f"OK. Ahora sé que debo recibir {largo_archivo} bytes")

# Ahora leemos el archivo por chunks, de máximo 4096 bytes.
while len(datos) < largo_archivo:
    # El último recv será probablemente más chico que 4096
    bytes_leer = min(4096, largo_archivo - len(datos))
    datos_recibidos = sock_cliente.recv(bytes_leer)
    # Recordemos que el método recv, entrega una cantidad máxima, pero
no nos asegura que nos
    # entregue los 4096 bytes. Es por esto, que la cantidad de bytes
que hemos recibido en
    # total, se deben ver siempre en función de lo que retornó el
método recv, y no lo que
    # le entregamos como parámetro
    bytes_leidos += len(datos_recibidos)
    print(
        f"He recibido {len(datos_recibidos)} bytes en el último recv.
Van {bytes_leidos} en total.")
    datos.extend(datos_recibidos)

print(f"¡Listo! He recibido {len(datos)} bytes")
```

```python
# Guardamos la información en un archivo.
with open('files/recibido.bin', 'wb') as binfile:
    binfile.write(datos)

print("¡Archivo guardado!")
# Le enviamos una respuesta a la contraparte.
sock_cliente.sendall("Gracias.".encode('utf-8'))

# Cerramos los sockets.
sock_cliente.close()
sock.close()

# Emisor mejorado del archivo.
import socket


host_receptor = 'localhost'
puerto_receptor = 12345

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Nos conectamos al receptor del archivo, que ya debería estar
escuchando.
sock.connect((host_receptor, puerto_receptor))
print("Conexión establecida.")

# Leemos el archivo y lo enviamos.
with open('files/enviar.bin', 'rb') as binfile:
    datos = binfile.read()
    largo_archivo = len(datos)
    print(f"Voy a enviar {largo_archivo} bytes")
    # 1. Enviar el tamaño del archivo.
    # (int.to_bytes transforma un entero en una cantidad de bytes por
el primer parámetro)
    sock.sendall(largo_archivo.to_bytes(4, byteorder='big'))

    # 2. Enviar el archivo. Para esto haremos uso del método send en
vez de sendall, para asegurarnos
    # que todo se envíe correctamente
    bytes_enviados = 0
    while bytes_enviados < largo_archivo:
        # Enviaremos el mensaje cortado desde el último punto
efectivamente enviado.
        bytes_enviados += sock.send(datos[bytes_enviados:])


print("¡Archivo enviado!")

# Imprimimos lo que nos responda la contraparte.
print("Respuesta:", sock.recv(4096).decode('utf-8'))
```

```
# Cerramos el socket.
sock.close()
```

Ahora podemos ver que funciona correctamente.

Servidor con manejo de múltiples clientes en forma concurrente

En la práctica, lo más probable es que tengamos que manejar múltiples usuarios de forma concurrente en el servidor, y además enviar paquetes de datos de tamaños arbitrarios. A continuación, describiremos cómo incorporaremos estas modificaciones.

El manejo de varios usuarios simultáneamente se logra mediante el uso de *threads*. Cada vez que se ejecutan los métodos `socket.accept()` y `socket.recv()` los *threads* donde ellos son invocados se bloquean hasta aceptar una nueva conexión (en el caso del primero), y recibir datos (en el caso del segundo). Para evitar este comportamiento tendremos que crear un *thread* encargado de:

- Aceptar nuevos clientes;
- Y, cada vez que se acepte un cliente, crear un *thread* nuevo que se ocupe de escuchar y enviar información al nuevo cliente conectado.

Además, implementaremos lo aprendido sobre cómo enviar o recibir mensajes de tamaño arbitrario. Definamos a continuación una clase `Client` donde ejemplificaremos las modificaciones mencionadas.

```
import socket
import threading


class Client:
    """
    Maneja toda la comunicación desde el lado del cliente.

    Implementa el esquema de comunicación donde los primeros 4 bytes
de cada
    mensaje indicarán el largo del mensaje enviado.
    """

    def __init__(self, port: int, host: str) -> None:
        print("Inicializando cliente...")

        self.host = host
        self.port = port
        self.socket_client = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
```

```
        try:
            self.connect_to_server()
            self.listen()
            self.repl()
        except ConnectionError:
            print("Conexión terminada.")
            self.socket_client.close()
            exit()

    def connect_to_server(self) -> None:
        """Crea la conexión al servidor."""

        self.socket_client.connect((self.host, self.port))
        print("Cliente conectado exitosamente al servidor.")

    def listen(self) -> None:
        """
        Inicializa el thread que escuchará los mensajes del servidor.

        Es útil hacer un thread diferente para escuchar al servidor,
        ya que de esa forma podremos tener comunicación asíncrona con
este.
        Luego, el servidor nos podrá enviar mensajes sin necesidad de
        iniciar una solicitud desde el lado del cliente.
        """

        thread = threading.Thread(target=self.listen_thread,
daemon=True)
        thread.start()

    def send(self, msg: str) -> None:
        """
        Envía mensajes al servidor.

        Implementa el mismo protocolo de comunicación que mencionamos;
        es decir, agregar 4 bytes al principio de cada mensaje
        indicando el largo del mensaje enviado.
        Luego, puedes elegir entre usar send() en un ciclo, o un solo
sendall().
        En este ejemplo, solo usamos un sendall(), pero puedes
cambiarlo por
        el ciclo del ejemplo anterior para asegurar que toda la
información
        se enviará correctamente.
        """

        msg_bytes = msg.encode()
        msg_length = len(msg_bytes).to_bytes(4, byteorder='big')
        self.socket_client.sendall(msg_length + msg_bytes)
```

```
    def listen_thread(self) -> None:
        while True:
            response_bytes_length = self.socket_client.recv(4)
            response_length = int.from_bytes(
                response_bytes_length, byteorder='big')
            response = bytearray()

            # Recibimos datos hasta que alcancemos la totalidad de los
datos
            # indicados en los primeros 4 bytes recibidos.
            while len(response) < response_length:
                read_length = min(4096, response_length -
len(response))
                response.extend(self.socket_client.recv(read_length))

            print(f"{response.decode()}\n>>> ", end='')

    def repl(self) -> None:
        """
        Captura el input del usuario.

        Lee mensajes desde el terminal y después se pasan a
`self.send()`.
        """

        print("------ Consola ------\n>>> ", end='')
        while True:
            msg = input()
            self.send(msg)


if __name__ == "__main__":
    port = 8080
    host = 'localhost'

    client = Client(port, host)
```

Ahora implementaremos la clase `Server` que se encargará de controlar toda la lógica en el lado del servidor:

```
import socket
import threading


class Server:
    def __init__(self, port: int, host: str) -> None:
        print("Inicializando servidor...")

        self.host = host
        self.port = port
```

```
        self.socket_server = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        self.bind_and_listen()
        self.accept_connections()

    def bind_and_listen(self) -> None:
        """
        Enlaza el socket creado con el host y puerto indicado.

        Primero, se enlaza el socket y luego queda esperando
        por conexiones entrantes.
        """
        self.socket_server.bind((self.host, self.port))
        self.socket_server.listen()
        print(f"Servidor escuchando en {self.host}:{self.port}...")

    def accept_connections(self) -> None:
        """
        Inicia el thread que aceptará clientes.

        Aunque podríamos aceptar clientes en el thread principal de la
        instancia, es útil hacerlo en un thread aparte. Esto nos
        permitirá realizar la lógica en la parte del servidor sin
dejar
        de aceptar clientes. Por ejemplo, seguir procesando archivos.
        """
        thread =
threading.Thread(target=self.accept_connections_thread)
        thread.start()

    def accept_connections_thread(self) -> None:
        """
        Es arrancado como thread para aceptar clientes.

        Cada vez que aceptamos un nuevo cliente, iniciamos un
        thread nuevo encargado de manejar el socket para ese cliente.
        """
        print("Servidor aceptando conexiones...")

        while True:
            client_socket, _ = self.socket_server.accept()
            listening_client_thread = threading.Thread(
                target=self.listen_client_thread,
                args=(client_socket, ),
                daemon=True)
            listening_client_thread.start()

    @staticmethod
    def send(value: any, sock: socket.socket) -> None:
        """
```

```
        Envía mensajes hacia algún socket cliente.

        Debemos implementar en este método el protocolo de
comunicación
        donde los primeros 4 bytes indicarán el largo del mensaje.
        Luego, puedes elegir entre usar send() en un ciclo, o un solo
sendall().
        En este ejemplo, solo usamos un sendall(), pero puedes
cambiarlo por
        el ciclo del ejemplo anterior para asegurar que toda la
información
        se enviará correctamente.
        """
        stringified_value = str(value)
        msg_bytes = stringified_value.encode()
        msg_length = len(msg_bytes).to_bytes(4, byteorder='big')
        sock.sendall(msg_length + msg_bytes)

    def listen_client_thread(self, client_socket: socket.socket) ->
None:
        """
        Es ejecutado como thread que escuchará a un cliente en
particular.

        Implementa las funcionalidades del protocolo de comunicación
        que permiten recuperar la informacion enviada.
        """
        print(f"{client_socket.getpeername()} Nuevo cliente conectado
al servidor")

        while True:
            response_bytes_length = client_socket.recv(4)
            response_length = int.from_bytes(
                response_bytes_length, byteorder='big')
            response = bytearray()

            while len(response) < response_length:
                read_length = min(4096, response_length -
len(response))
                response.extend(client_socket.recv(read_length))

            received = response.decode()

            if received != "":
                # El método `self.handle_command()` debe ser definido.
                # Este realizará toda la lógica asociado a los
mensajes
                # que llegan al servidor desde un cliente en
particular.
                # Se espera que retorne la respuesta que el servidor
```

```
                # debe enviar hacia el cliente.
                response = self.handle_command(received,
client_socket)
                self.send(response, client_socket)

    def handle_command(self, received: str, client_socket:
socket.socket) -> str:
        print(f"{client_socket.getpeername()} Comando recibido:",
received)
        # Este método debería ejecutar la acción y enviar una
respuesta.
        return "Acción asociada a " + received


if __name__ == "__main__":
    port = 8080
    host = 'localhost'

    server = Server(port, host)
```

Las clases descritas en los códigos anteriores definen entidades que podrán ser usadas eficientemente a través de *threads* para comunicación asíncrona. Es recomendable mantener las funcionalidades de estas clases sólo destinadas a funciones de *networking*, i.e., solo para efectuar la comunicación entre el servidor y los clientes. Será natural que intentemos agregar código que permita manejar la lógica en el cliente o el servidor de acuerdo a los requerimientos de nuestro problema, lo que tendrá como consecuencia la generación de código muy **difícil de leer y mantener**.

Por lo tanto, recomendamos fuertemente efectuar la implementación de la lógica en otras clases (*backend*). Y destinar las clases Cliente y Servidor a solo funciones encargadas de la comunicación.

1Agradecemos al ex-ayudante del curso, Rodolfo Palma, por su colaboración para la elaboración del ejemplo práctico de este material.

Cliente conectado a más de un servidor

Aunque no es tan común como la versión inversa (un servidor con múltiples clientes), puede ocurrir que nuestro cliente necesite conectarse a más de un servidor al mismo tiempo. Esto podría deberse a múltiples razones, como que cada servidor pertenece a un servicio diferente o a pesar de ser del mismo servicio, se encargue de diferentes funcionalidades de un producto.

La forma de trabajar con más de un servidor es similar a la del ejemplo anterior en el sentido de que necesitaremos incorporar un thread por conexión, sin embargo la diferencia recae en que esta vez no podemos usar el mismo socket para ambas conexiones, ya que el socket es creado por los servidores y no los clientes. Para este ejemplo deberemos entablar una conexión individual a cada servidor, y nuestro programa deberá manejar a qué servidor enviar la información según corresponda, y cómo procesar la información recibida dependiendo de qué servidor la envió.

Nuevamente, para ejecutar este ejemplo se recomienda crear los archivos `servidor_1.py`, `servidor_2.py` y `cliente.py`, en los cuales se debe pegar el código presentado en estas celdas.

Partiremos instanciando dos servidores tal como se ha hecho anteriormente. El código de ambos servidores es el mismo, solo usando prints para diferenciarlos. Otro detalle a notar es que cada uno de los servidores tiene su propio puerto, ya que esta es la forma con la que el cliente sabrá cómo diferenciar a cada uno, especialmente cuando tienen la misma IP (al ser ejecutados desde el mismo computador).

```
# Crea un archivo servidor_1.py con el código de esta celda y
ejecútalo.
import socket
import threading


class Server:
    def __init__(self, port: int, host: str) -> None:
        print("Inicializando servidor 1...")

        self.host = host
        self.port = port
        self.socket_server = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        self.bind_and_listen()
        self.accept_connections()

    def bind_and_listen(self) -> None:
        self.socket_server.bind((self.host, self.port))
        self.socket_server.listen()
        print(f"Servidor 1 escuchando en {self.host}:{self.port}...")

    def accept_connections(self) -> None:
        thread =
threading.Thread(target=self.accept_connections_thread)
        thread.start()

    def accept_connections_thread(self) -> None:
        print("Servidor 1 aceptando conexiones...")

        while True:
            client_socket, _ = self.socket_server.accept()
            listening_client_thread = threading.Thread(
                target=self.listen_client_thread,
                args=(client_socket, ),
                daemon=True)
            listening_client_thread.start()

    @staticmethod
    def send(value: any, sock: socket.socket) -> None:
```

```
        stringified_value = str(value)
        msg_bytes = stringified_value.encode()
        sock.sendall(msg_bytes)

    def listen_client_thread(self, client_socket: socket.socket) ->
None:
        print("Servidor 1 conectado a un nuevo cliente...")

        while True:
            response = client_socket.recv(4096)

            received = response.decode()

            if received != "":
                response = self.handle_command(received,
client_socket)
                self.send(response, client_socket)

    def handle_command(self, received: str, client_socket:
socket.socket) -> str:
        print("Comando recibido por el servidor 1:", received)
        # Este método debería ejecutar la acción y enviar una
respuesta.
        return "Acción del servidor 1 asociada a " + received


if __name__ == "__main__":
    # Nótese que cada servidor debe definir su propio puerto distinto.
    port = 8080
    host = 'localhost'

    server = Server(port, host)

# Crea un archivo servidor_2.py con el código de esta celda y
ejecútalo.
import socket
import threading


class Server:
    def __init__(self, port: int, host: str) -> None:
        print("Inicializando servidor 2...")

        self.host = host
        self.port = port
        self.socket_server = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        self.bind_and_listen()
        self.accept_connections()

    def bind_and_listen(self) -> None:
```

```
        self.socket_server.bind((self.host, self.port))
        self.socket_server.listen()
        print(f"Servidor 2 escuchando en {self.host}:{self.port}...")

    def accept_connections(self) -> None:
        thread =
threading.Thread(target=self.accept_connections_thread)
        thread.start()

    def accept_connections_thread(self) -> None:
        print("Servidor 2 aceptando conexiones...")

        while True:
            client_socket, _ = self.socket_server.accept()
            listening_client_thread = threading.Thread(
                target=self.listen_client_thread,
                args=(client_socket, ),
                daemon=True)
            listening_client_thread.start()

    @staticmethod
    def send(value: any, sock: socket.socket) -> None:
        stringified_value = str(value)
        msg_bytes = stringified_value.encode()
        sock.sendall(msg_bytes)

    def listen_client_thread(self, client_socket: socket.socket) ->
None:
        print("Servidor 2 conectado a un nuevo cliente...")

        while True:
            response = client_socket.recv(4096)

            received = response.decode()

            if received != "":
                response = self.handle_command(received,
client_socket)
                self.send(response, client_socket)

    def handle_command(self, received: str, client_socket:
socket.socket) -> str:
        print("Comando recibido por el servidor 2:", received)
        # Este método debería ejecutar la acción y enviar una
respuesta.
        return "Acción del servidor 2 asociada a " + received


if __name__ == "__main__":
    # Nótese que cada servidor debe definir su propio puerto distinto.
```

```
    port = 8081
    host = 'localhost'

    server = Server(port, host)
```

Ahora debemos instanciar al cliente.

En este caso, notaremos que a diferencia de los ejemplos anteriores, deberemos generar dos hilos independientes entre sí, donde cada uno entabla y almacena una conexión con uno de los servidores. Entonces, cada vez que queramos interactuar con uno de los servidores, deberemos usar el *socket* relacionado a dicho servidor para enviar el mensaje.

```
# Crea un archivo cliente.py con el código de esta celda y ejecútalo
# (una vez ya hayas ejecutado los dos servidores anteriores).
import socket
import threading
import time


class Client:
    """
    Maneja toda la comunicación desde el lado del cliente.
    """

    def __init__(self, datos_servidor_1: dict, datos_servidor_2: dict)
-> None:
        print("Inicializando cliente...")

        self.datos_servidor_1 = datos_servidor_1
        self.datos_servidor_2 = datos_servidor_2
        # Tenemos que crear un socket para cada servidor, ya que
necesitarán su propia conexión.
        # Usamos un dict para mejor escalabilidad (tener todos los
servidores en 1 variable)
        self.sockets = {
            1: socket.socket(
                socket.AF_INET, socket.SOCK_STREAM),
            2: socket.socket(
                socket.AF_INET, socket.SOCK_STREAM)
        }

        try:
            self.connect_to_server()
            self.listen()
            self.repl()
        except ConnectionError:
            print("Conexión terminada.")
            self.disconnect_servers()
            exit()
```

```
    def connect_to_server(self) -> None:
        """
        Crea la conexión al servidor.
        Con más de un servidor, debemos repetir los comandos de
conexión para
        cada servidor.
        """

        self.sockets[1].connect(
            (self.datos_servidor_1['host'],
self.datos_servidor_1['port']))

        self.sockets[2].connect(
            (self.datos_servidor_2['host'],
self.datos_servidor_2['port']))

        print("Cliente conectado exitosamente a ambos servidores.")

    def disconnect_servers(self) -> None:
        for socket in self.sockets.values():
            socket.close()

    def listen(self) -> None:
        """
        Inicializa los threads que escucharán los mensajes de los
servidores.

        Para que los servidores no se bloqueen entre sí, debemos tener
un thread por conexión
        que se encargue de escuchar exclusivamente a ese servidor.
        """

        thread_1 = threading.Thread(
            target=self.listen_thread, args=(1,), daemon=True)
        thread_2 = threading.Thread(
            target=self.listen_thread, args=(2,), daemon=True)
        thread_1.start()
        thread_2.start()

    def send(self, server: int, msg: str) -> None:
        """
        Envía mensajes al servidor.

        Debemos escoger qué mensaje mandar a qué servidor y usar su
socket correspondiente.
        """

        msg_bytes = msg.encode()
        self.sockets[server].sendall(msg_bytes)
```

```
    def listen_thread(self, servidor) -> None:
        """
        Función que escucha al servidor. Dependiendo de cuál eligamos,
hace receive de la
        información del socket elegido y lo identifica al hacer print.
        """
        while True:
            response = self.sockets[servidor].recv(4096)
            print(
                f"El siguiente mensaje fue enviado por el servidor
{servidor}:")
            print(f"{response.decode()}\n>>> ", end='')

    def repl(self) -> None:
        """
        Captura el input del usuario.

        Lee mensajes desde el terminal y después se pasan a
`self.send()`.
        """

        print("------ Consola ------\n>>> ", end='')
        while True:
            server = input(
                "Selecciona a que servidor deseas escribir (1, 2): ")
            msg = input("Escribe tu mensaje: ")
            self.send(int(server), msg)
            # Este sleep() es solo para que los textos en la consola
no se sobrepongan
            # Y el ejemplo se entienda mejor
            time.sleep(1)


if __name__ == "__main__":
    # Necesitamos saber los datos de ambos servidores
    datos_servidor_1 = {
        'port': 8080,
        'host': 'localhost',
    }

    datos_servidor_2 = {
        'port': 8081,
        'host': 'localhost',
    }

    client = Client(datos_servidor_1, datos_servidor_2)
```

Interacción entre clientes *Peer 2 Peer* (P2P)

Hasta ahora, hemos trabajado con arquitecturas centralizadas en donde siempre existe una diferencia clara entre Servidor (quien se encarga de procesar solicitudes, responder a clientes, y en generar administrar la red en caso de haber múltiples clientes) y Cliente (quienes solicitan o entregan información al servidor cuando lo necesitan). Sin embargo, no es el único tipo de arquitectura disponible.

En el contexto de redes, el concepto de *Peer 2 Peer* corresponde a un tipo de arquitectura no centralizada en donde dejamos de tener un servidor único que maneja gran parte de la carga o trabajo de la red, y solo poseemos clientes que se comunican entre sí, distribuyéndose la carga o trabajo de la red entre ellos. Ejemplos conocidos de arquitecturas P2P son las redes de transferencia de archivos (como cierta aplicación que no podemos mencionar pero... El que sabe, *sabe* ☠).

Para implementar la arquitectura P2P con *sockets* en Python, ya no tendremos archivo de servidor y cliente, sino que ambos serán clientes en el sentido de que ninguno tiene mayor privilegio que el otro al trabajar, sino que ambos son equivalentes en jerarquía. A diferencia de los ejemplos anteriores, como resultados tendremos un programa que para fines prácticos es al mismo tiempo un Servidor (ya que debe crear un *socket* en cierto puerto, para aceptar conexiones de otros clientes) y un Cliente (ya que debe conectarse a los *sockets* de otros clientes). Por lo que nuestras clases ahora deberán tener tanto los métodos de servidor como de cliente disponibles. Nuevamente, haremos uso de *threads* para que estos *sockets* no se bloqueen entre sí y podamos tenerlos trabajando en paralelo.

Por simpleza, tendremos en este ejemplo código que deberá ir en dos archivos, `cliente_1.py` y `cliente_2.py` que deben ejecutarse en ese orden para funcionar. Esto es solo para evitar el error de ejecutar en desorden y tratar de conectarnos a un *socket* de un programa que no esté corriendo todavía. En fines prácticos, si quieres añadir lógica adicional para manejar este posible problema, podrías condensar ambos archivos en un solo archivo `cliente.py` que use el mismo código (pero eso te lo dejamos como desafío 😉)

Vamos al ejemplo:

```
# Crea un archivo cliente_1.py con el código de esta celda y ejecútalo
primero.
import socket
import threading
import time


class Client_1:
    """
    Nuestro primer cliente Peer 2 Peer (Que es un server y cliente a
la vez).
    Este abre un servidor, acepta la conexión de otro cliente
(haciendo que tomemos el rol de server)
    Luego, se trata de conectar al servidor de ese cliente (haciendo
que tomemos el rol de cliente también).
    """
```

```
    def __init__(self, host: str, port: int, datos_otro_cliente: dict)
-> None:
        print("Inicializando cliente P2P (1)...")
        self.host = host
        self.port = port
        self.datos_otro_cliente = datos_otro_cliente

        # Tenemos que crear un socket para nosotros abrir y escuchar
conexiones como servidor,
        # Y un socket que usaremos para conectarnos al otro cliente
P2P como clientes de su Servidor.
        self.sockets = {
            'server': socket.socket(
                socket.AF_INET, socket.SOCK_STREAM),
            'client': socket.socket(
                socket.AF_INET, socket.SOCK_STREAM)
        }

        # Primero abriremos nuestro socket como servidor para aceptar
conexiones de otros clientes
        self.bind_and_listen()
        self.accept_connections()

    # Los siguientes 6 métodos sirven para realizar las acciones que
haría un servidor
    # INICIO MÉTODOS SERVIDOR
    def bind_and_listen(self) -> None:
        """
        Nos enlazamos a un host y puerto para escuchar conexiones
        """
        self.sockets['server'].bind((self.host, self.port))
        self.sockets['server'].listen()
        print(f"Cliente P2P (1) escuchando en {self.host}:
{self.port}...")

    def accept_connections(self) -> None:
        """
        Hacemos un thread solo para aceptar conexiones
        """
        thread =
threading.Thread(target=self.accept_connections_thread)
        thread.start()

    def accept_connections_thread(self) -> None:
        """
        Levantamos un thread por cliente que se quiere conectar a
nosotros.
        """
        print("Cliente P2P (1) aceptando conexiones...")
```

```
        while True:
            client_socket, _ = self.sockets['server'].accept()
            listening_client_thread = threading.Thread(
                target=self.listen_client_thread,
                args=(client_socket, ),
                daemon=True)
            listening_client_thread.start()
            # Como ya aceptamos al otro cliente, significa que este ya
está online.
            # Entonces, podemos conectarnos al servidor de este y
volvernos sus clientes
            try:
                self.connect_to_server()
                self.listen()
                self.repl()
            except ConnectionError:
                print("Conexión terminada.")
                self.sockets['client'].close()
                exit()

    def listen_client_thread(self, client_socket: socket.socket) ->
None:
        """
        Decodificamos y respondemos los mensajes de los clientes.
        """
        print("Cliente P2P (1) recibió a un nuevo cliente...")

        while True:
            response = client_socket.recv(4096)

            received = response.decode()

            if received != "":
                response = self.handle_command(received,
client_socket)
                self.answer_client(response, client_socket)

    def handle_command(self, received: str, client_socket:
socket.socket) -> str:
        print("Comando recibido por el Cliente P2P (1):", received)
        # Este método debería ejecutar la acción y enviar una
respuesta.
        return "Acción del Cliente P2P (1) asociada a " + received

    @staticmethod
    def answer_client(value: any, sock: socket.socket) -> None:
        """
        Enviamos la respuesta a un cliente en específico.
        En este caso, lo enviamos como si fueramos un servidor que
```

```
responde a su cliente.
        """
        stringified_value = str(value)
        msg_bytes = stringified_value.encode()
        sock.sendall(msg_bytes)

    # FIN MÉTODOS SERVIDOR

    # Los siguientes 5 métodos son los que necesitamos para actuar
como clientes.
    # INICIO MÉTODOS CLIENTE
    def connect_to_server(self) -> None:
        """
        Crea la conexión al otro cliente (que será nuestro server).
        """

        self.sockets['client'].connect(
            (self.datos_otro_cliente['host'],
self.datos_otro_cliente['port']))

        print("Cliente conectado exitosamente al otro cliente.")

    def listen(self) -> None:
        """
        Inicializa el thread que escuchará al otro cliente como si
fuera un servidor.
        """

        thread_listening_server = threading.Thread(
            target=self.listen_thread, daemon=True)
        thread_listening_server.start()

    def listen_thread(self) -> None:
        """
        Función que escucha al otro cliente como servidor.
        """
        while True:
            response = self.sockets['client'].recv(4096)
            print(
                f"El siguiente mensaje fue una respuesta por el otro
cliente P2P:")
            print(f"{response.decode()}\n>>> ", end='')

    def send(self, msg: str) -> None:
        """
        Envía mensajes al otro cliente iniciados por nosotros.
        En este caso, lo estamos enviando como si fueramos un cliente
de su servidor.
        """
```

```
        msg_bytes = msg.encode()
        self.sockets['client'].sendall(msg_bytes)

    def repl(self) -> None:
        """
        Captura el input del usuario para iniciar mensajes.

        Lee mensajes desde el terminal y después se pasan a
`self.send()`.
        """
        # Nuevamente, estos sleep() son solo para mantener orden de
los prints por fines didácticos.
        time.sleep(1)
        print("------ Consola P2P (1) ------\n>>> ", end='')
        while True:
            msg = input("Escribe tu mensaje al otro cliente: ")
            self.send(msg)
            time.sleep(1)

    # FIN MÉTODOS CLIENTE


if __name__ == "__main__":
    # Necesitamos saber los datos nuestros para levantar un server
interno
    host = 'localhost'
    port = 8080
    # Y los datos del otro cliente con el que nos conectaremos
    datos_otro_cliente = {
        'port': 8081,
        'host': 'localhost',
    }

    client = Client_1(host, port, datos_otro_cliente)

# Crea un archivo cliente_2.py con el código de esta celda y ejecútalo
segundo.
import socket
import threading
import time


class Client_2:
    """
    Nuestro segundo cliente Peer 2 Peer (Que es un server y cliente a
la vez).
    Este primero abre un servidor y espera conexiones.
    Luego, se conecta al Cliente_1 como si nosotros fueramos un
Cliente de su Servidor.
    Posteriormente, aceptamos la solicitud de Cliente_1 para
```

```
conectarse a nosotros como si
    fueramos un servidor, cerrando el ciclo y volviendo nuestra
comunicación P2P.
    """

    def __init__(self, host: str, port: int, datos_otro_cliente: dict)
-> None:
        print("Inicializando cliente P2P (2)...")
        self.host = host
        self.port = port
        self.datos_otro_cliente = datos_otro_cliente

        # Tenemos que crear un socket para nosotros abrir y escuchar
conexiones como servidor,
        # Y un socket que usaremos para conectarnos al otro cliente
P2P como clientes de su Servidor.
        self.sockets = {
            'server': socket.socket(
                socket.AF_INET, socket.SOCK_STREAM),
            'client': socket.socket(
                socket.AF_INET, socket.SOCK_STREAM)
        }

        # Primero abriremos nuestro socket como servidor para aceptar
conexiones de otros clientes
        self.bind_and_listen()
        self.accept_connections()

        # Luego, como asumimos que el otro servidor ya está levantado,
inmediatamente nos conectaremos a
        # Este como clientes
        try:
            self.connect_to_server()
            self.listen()
            self.repl()
        except ConnectionError:
            print("Conexión terminada.")
            self.sockets['client'].close()
            exit()

    # Los siguientes 6 métodos sirven para realizar las acciones que
haría un servidor
    # INICIO MÉTODOS SERVIDOR

    def bind_and_listen(self) -> None:
        """
        Nos enlazamos a un host y puerto para escuchar conexiones
        """
        self.sockets['server'].bind((self.host, self.port))
        self.sockets['server'].listen()
```

```
        print(f"Cliente P2P (2) escuchando en {self.host}:
{self.port}...")

    def accept_connections(self) -> None:
        """
        Hacemos un thread solo para aceptar conexiones
        """
        thread =
threading.Thread(target=self.accept_connections_thread)
        thread.start()

    def accept_connections_thread(self) -> None:
        """
        Levantamos un thread por cliente que se quiere conectar a
nosotros.
        """
        print("Cliente P2P (2) aceptando conexiones...")

        while True:
            client_socket, _ = self.sockets['server'].accept()
            listening_client_thread = threading.Thread(
                target=self.listen_client_thread,
                args=(client_socket, ),
                daemon=True)
            listening_client_thread.start()

    def listen_client_thread(self, client_socket: socket.socket) ->
None:
        """
        Decodificamos y respondemos los mensajes de los clientes.
        """
        print("Cliente P2P (2) recibió a un nuevo cliente...")

        while True:
            response = client_socket.recv(4096)

            received = response.decode()

            if received != "":
                response = self.handle_command(received,
client_socket)
                self.answer_client(response, client_socket)

    def handle_command(self, received: str, client_socket:
socket.socket) -> str:
        print("Comando recibido por el Cliente P2P (2):", received)
        # Este método debería ejecutar la acción y enviar una
respuesta.
        return "Acción del Cliente P2P (2) asociada a " + received
```

```
    @staticmethod
    def answer_client(value: any, sock: socket.socket) -> None:
        """
        Enviamos la respuesta a un cliente en específico.
        En este caso, lo enviamos como si fueramos un servidor que
responde a su cliente.
        """
        stringified_value = str(value)
        msg_bytes = stringified_value.encode()
        sock.sendall(msg_bytes)

    # FIN MÉTODOS SERVIDOR

    # Los siguientes 5 métodos son los que necesitamos para actuar
como clientes.
    # INICIO MÉTODOS CLIENTE
    def connect_to_server(self) -> None:
        """
        Crea la conexión al otro cliente.
        """

        self.sockets['client'].connect(
            (self.datos_otro_cliente['host'],
self.datos_otro_cliente['port']))

        print("Cliente conectado exitosamente al otro cliente.")

    def listen(self) -> None:
        """
        Inicializa el thread que escuchará al otro cliente como si
fuera un servidor.
        """
        thread_listening_server = threading.Thread(
            target=self.listen_thread, daemon=True)
        thread_listening_server.start()

    def listen_thread(self) -> None:
        """
        Función que escucha al otro cliente como servidor.
        """
        while True:
            response = self.sockets['client'].recv(4096)
            print(
                f"El siguiente mensaje fue una respuesta por el otro
cliente P2P:")
            print(f"{response.decode()}\n>>> ", end='')

    def send(self, msg: str) -> None:
        """
        Envía mensajes al otro cliente.
```

```
        En este caso, lo estamos enviando como si fueramos un cliente
de su servidor.
        """

        msg_bytes = msg.encode()
        self.sockets['client'].sendall(msg_bytes)

    def repl(self) -> None:
        """
        Captura el input del usuario para iniciar mensajes.

        Lee mensajes desde el terminal y después se pasan a
`self.send()`.
        """
        # Nuevamente, estos sleep() son solo para mantener orden de
los prints por fines didácticos.
        time.sleep(1)
        print("------ Consola P2P (2) ------\n>>> ", end='')
        while True:
            msg = input("Escribe tu mensaje al otro cliente: ")
            self.send(msg)
            time.sleep(1)

    # FIN MÉTODOS CLIENTE


if __name__ == "__main__":
    # Necesitamos saber los datos nuestros para levantar un server
interno
    host = 'localhost'
    port = 8081
    # Y los datos del otro cliente con el que nos conectaremos
    datos_otro_cliente = {
        'port': 8080,
        'host': 'localhost',
    }

    client = Client_2(host, port, datos_otro_cliente)
```

Bastante código, ¿no? Puede que sea un poco complicado al inicio, pero al final de cuentas todos los ejemplos se resumen en:

- Quien debería ser un servidor, quien debería ser cliente. (Recordando que podemos tener múltiples servidores y clientes, y clases que pueden ser ambas)
- Generar un *thread* por cada *socket* que instanciemos.
- Generar un *thread* por cada cliente que se conecte a nosotros.
- Evaluar la necesidad de *threads* extras (para interactuar con la terminal, por ejemplo, u otros procesos que sean necesarios).

Con estos ejemplos, deberías poder generar distintas combinaciones de arquitecturas para tus programas.